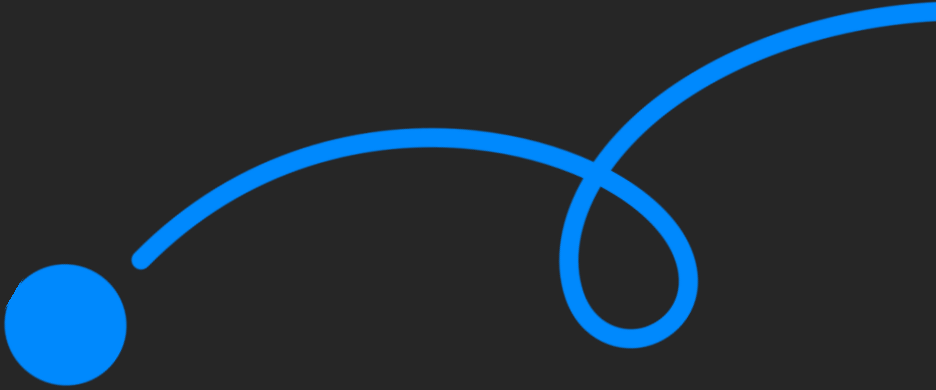




코멘토 실무PT 클래스

반도체 SW 엔지니어를 위한 새로운 SoC 위에 리눅스 디바이스 드라이버 포팅하기

멘티님, 반갑습니다!

클래스는 20:00에 시작됩니다.

출석 확인을 위해 프로필은 **실명**으로 설정해주세요.

출석을 확인합니다.



김대용

김진영

원종진

조현경

최승원

최민우

최강주

지난 세션은 어떠셨나요.

- 1 지난 주차에 어떤걸 공부했었는지 짧게 리뷰 해봐요.
- 2 복습을 진행하면서 궁금한 점이 있다면 같이 토의해봐요.
- 3 과제를 진행하면서 어려웠던 점을 알려주세요.
- 4 멘티 여러분이 작성한 과제를 다같이 살펴보도록 해요.

시작하기에 앞서

1

수업 진행을 하며
멘티님들의
이해도를 체크하며
진행할 예정입니다.

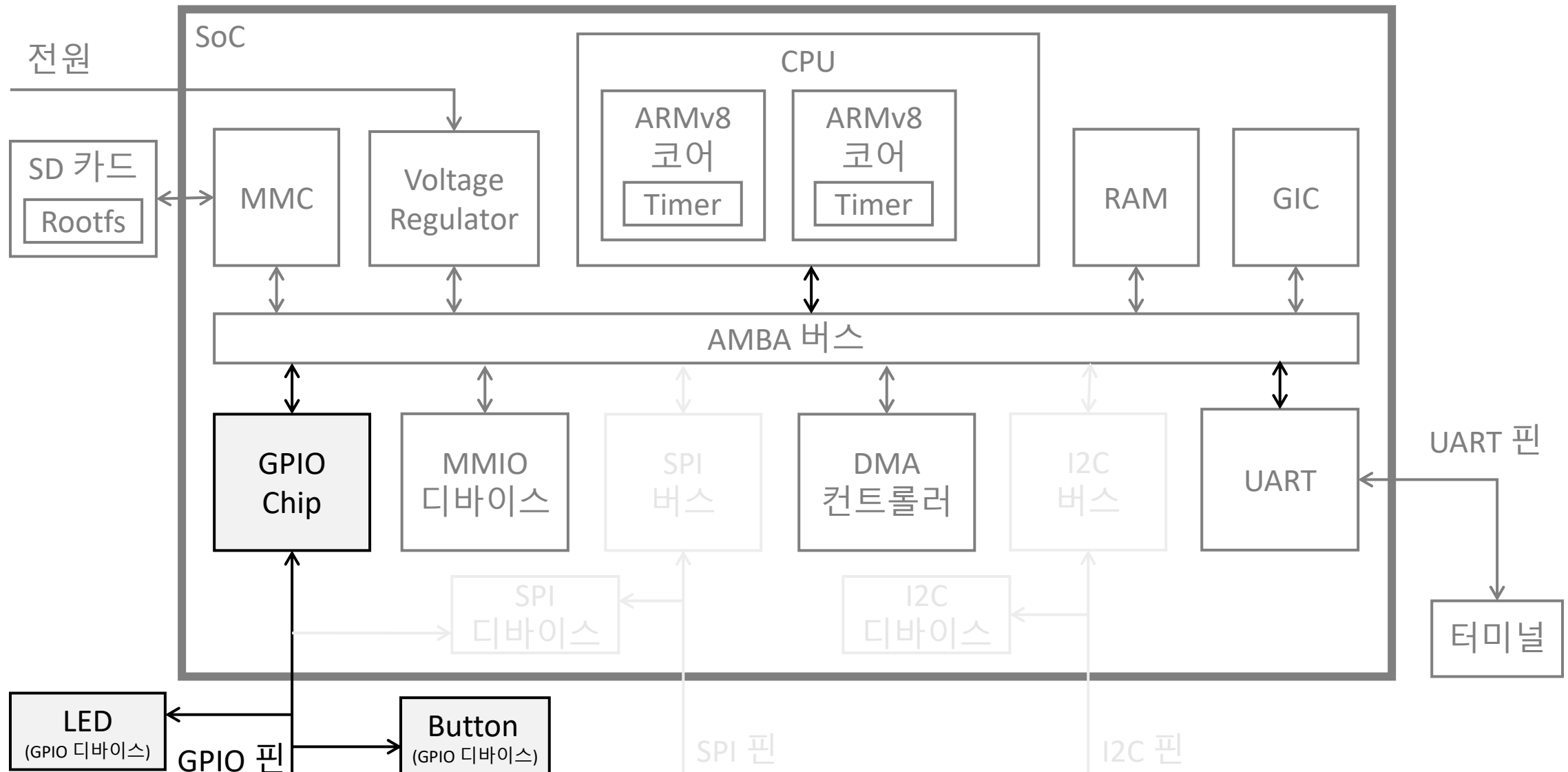
2

이해가 되지 않거나,
궁금한 내용은
채팅으로 언제든지
남겨주세요.

3

평균 50분 라이브를
진행하고, 10분의
쉬는 시간으로
진행 됩니다.

이번 수업에서 진행할 내용



01 GPIO 이해하기

02 새로운 GPIO 하드웨어 만들기

03 libgpiod 툴로 GPIO 테스트하기

04 GPIO 하드웨어 드라이버 만들기

05 PWM 이해하기

Chapter. 1

GPIO 이해하기

GPIO (General Purpose Input / Output) 란?

지난 강의까지는 모두 SoC 내부에서 벌어지는 일 (MMC 하드웨어 제외)

- 이번 주 강의부터는 SoC 외부와 소통을 할 수 있는 GPIO, SPI, I2C에 대해 다룸

SoC 내부의 신호를 외부로 또는 외부 신호를 내부로 전달하는 가장 간단한 형태

- 하드웨어와 소프트웨어 사이를 가르는 관문

1 (High)와 0 (Low)의 bit 형태로 값을 외부로 전달하거나 내부로 읽어 올 수 있음

- 전압이 있는 상태는 High, 전압이 없는 상태는 Low (1강에서 잠시 설명했었음)

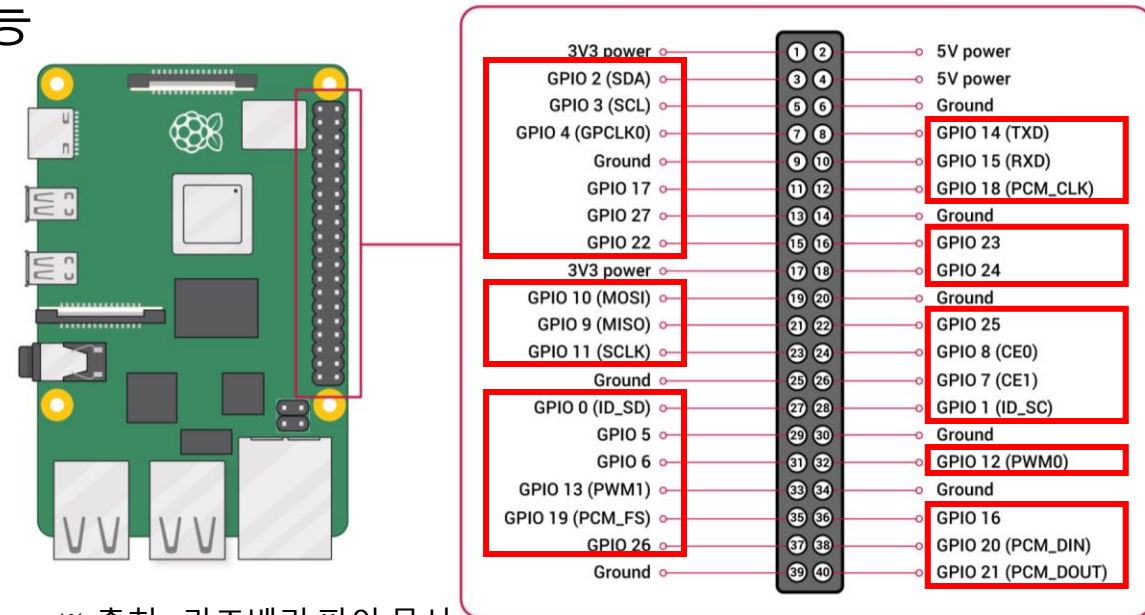
실제 SoC에서 GPIO의 형태

라즈베리 파이에 사용되는 SoC(Broadcom社 BCM27xx)는 많은 수의 GPIO 핀을 지원

- 라즈베리 파이는 IoT(Internet of Things)를 위해 많이 사용되는 저가형 보드
- 외부로 신호를 보내거나 외부 신호를 받아야 하는 경우가 많음

- 외부로 나온 핀 헤더를 사용하여 GPIO 사용 가능
- 스위치의 신호를 받거나 LED를 켜는 등의 예제

➤ 전압이 있는 상태와 없는 상태란
정확히 어떤 것을 의미하는 걸 까?



※ 출처 : 라즈베리 파이 문서

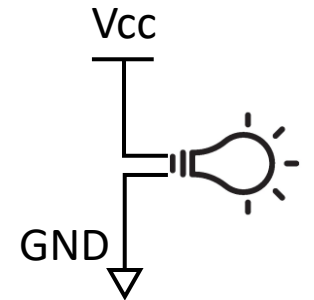
SoC 개발자가 알면 좋은 회로도 보는 법

모든 전기를 사용하는 장치는 Vcc와 GND (Ground)가 존재

- 전기는 순환이 되어야 일을 할 수 있음, +극에서 전기가 나가 -극으로 나오는 것
- Vcc는 +극을 GND는 -극을 의미
 - 엄밀히 말하면, GND는 접지이며 일반적으로 -극에 접지를 연결
 - 이것까지는 소프트웨어 하는 사람이 알 필요는 없음



실제 구조대로 그리면 복잡하므로 VCC는 막대로 GND는 삼각형으로 표시



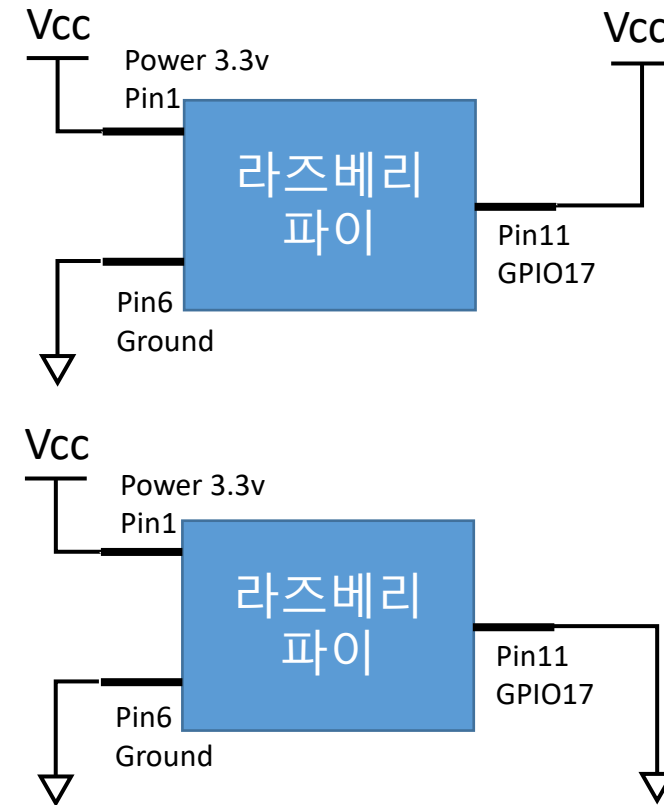
GPIO 입력의 물리적인 구조

전압이 있는 상태, High는 Vcc가 연결된 상태

- GPIO17의 입력을 읽으면 1

전압이 없는 상태, Low는 GND가 연결된 상태

- GPIO17의 입력을 읽으면 0

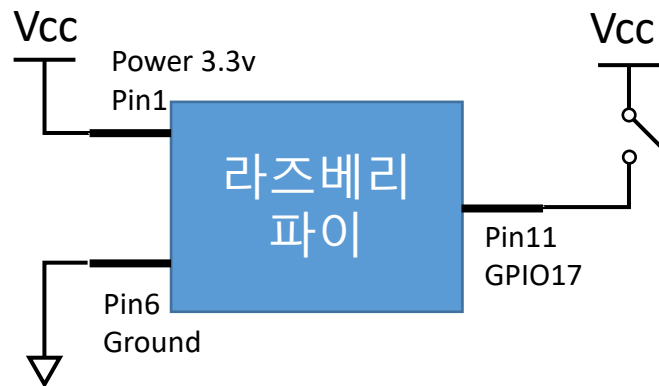


NC (Not connected)

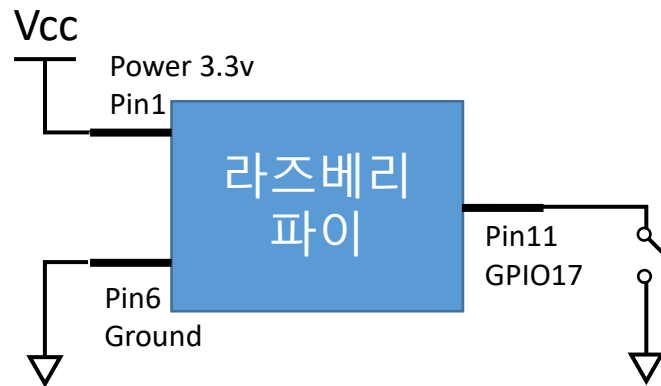
VCC도 GND도 연결이 되어있지 않다면? NC (Not Connected) 상태

- High도 Low도 아니므로 어떤 값이 들어올지 예측 불가 – 플로팅(Floating) 상태

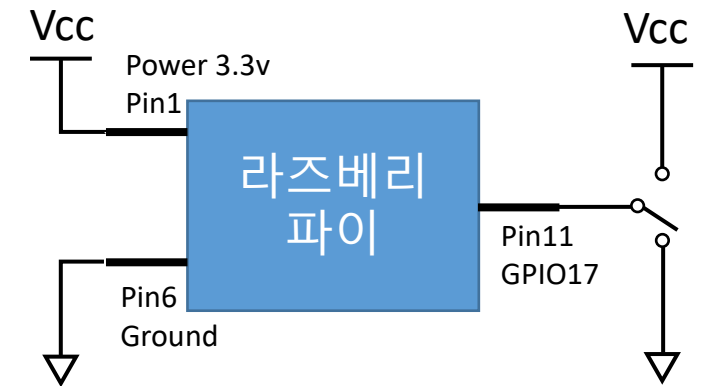
스위치 신호는 어떻게 받아야 할까?



스위치 닫으면 High,
스위치 열면 NC



스위치 닫으면 Low,
스위치 열면 NC

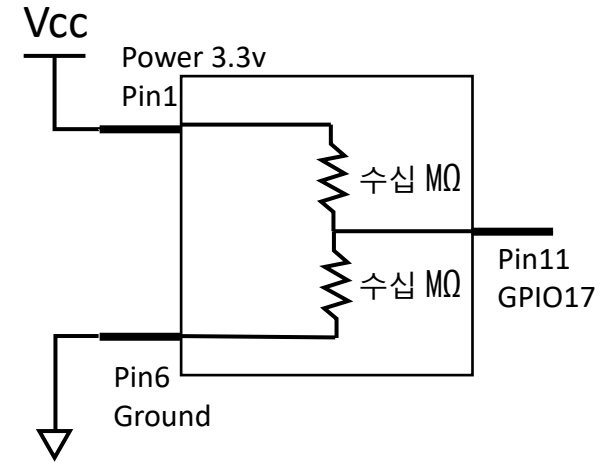


VCC 연결되면 High, GND 연결되면 Low
스위치 바꾸는 중에는 NC

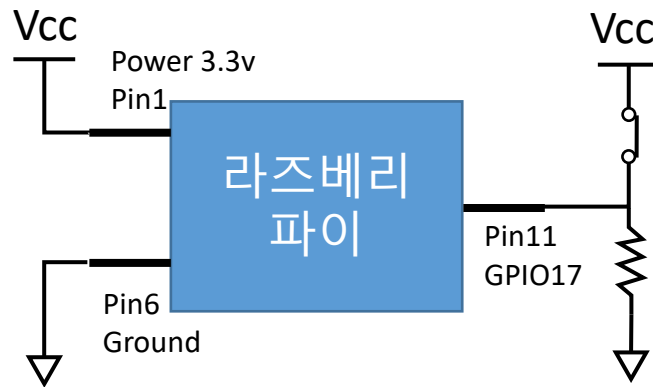
풀다운 (Pull-down) 저항

스위치 신호 설계시 GPIO 내부 저항은 이론상 ∞ 인 점을 이용

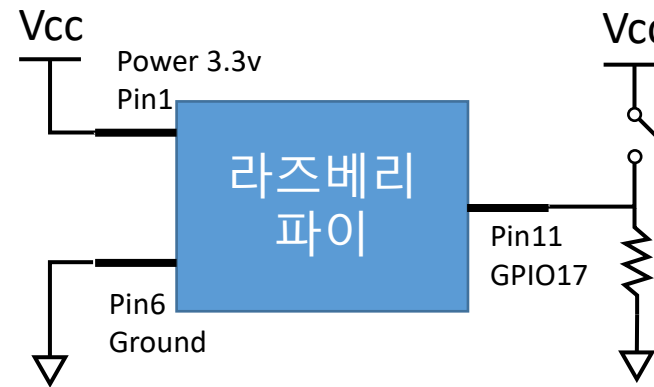
- 라즈베리 파이의 GPIO 내부 저항은 3.3V에서 최소 66M Ω 이상



풀다운 저항 : VCC 쪽에 스위치를 달았을 때, NC를 막기 위해 GND와 연결된 수k Ω 의 저항



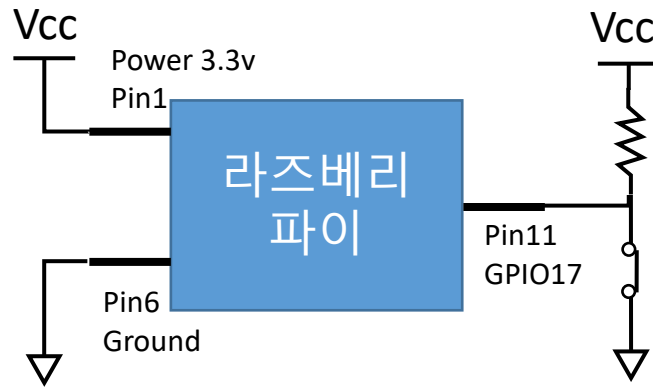
VCC에 바로 연결, Pin11에 전압이 있는 상태



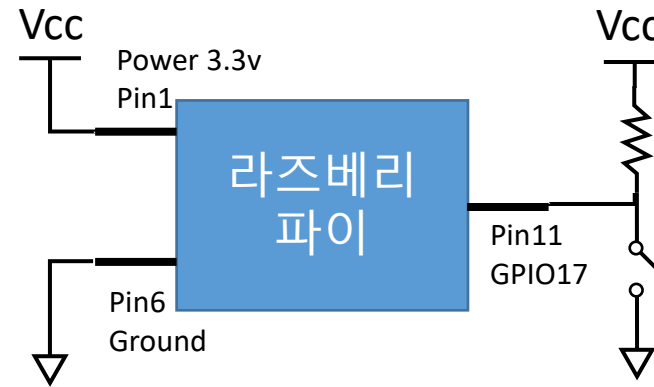
GND에 저항을 통해 연결, 내부저항이 커서 외부저항 효과는 무시 가능, Pin11에 전압이 없는 상태

풀업 (Pull-up) 저항

풀업 저항 : GND 쪽에 스위치를 달았을 때, NC를 막기 위해 VCC와 연결된 수k Ω 의 저항



GND에 바로 연결, Pin11에 전압이 없는 상태



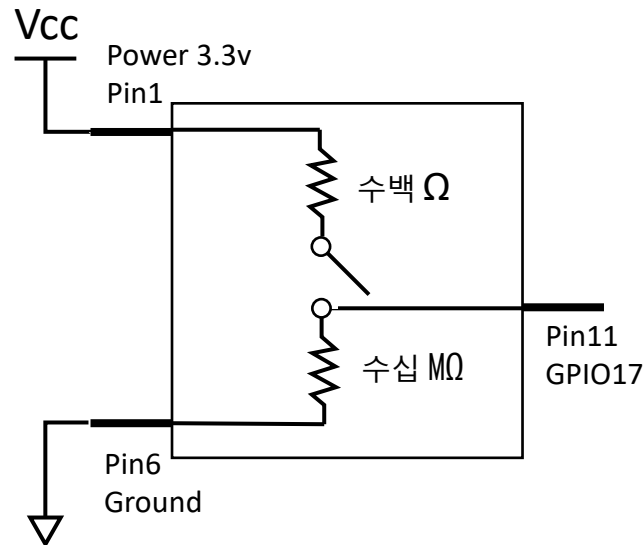
VCC에 저항을 통해 연결,
내부저항이 커서 외부저항 효과는 무시 가능
Pin11에 전압이 있는 상태

GPIO 출력의 물리적인 구조

SoC 내부의 스위치를 통해 VCC와 연결하거나 GND와 연결됨

일반적으로 GPIO 출력의 최대 전류는 10~20mA

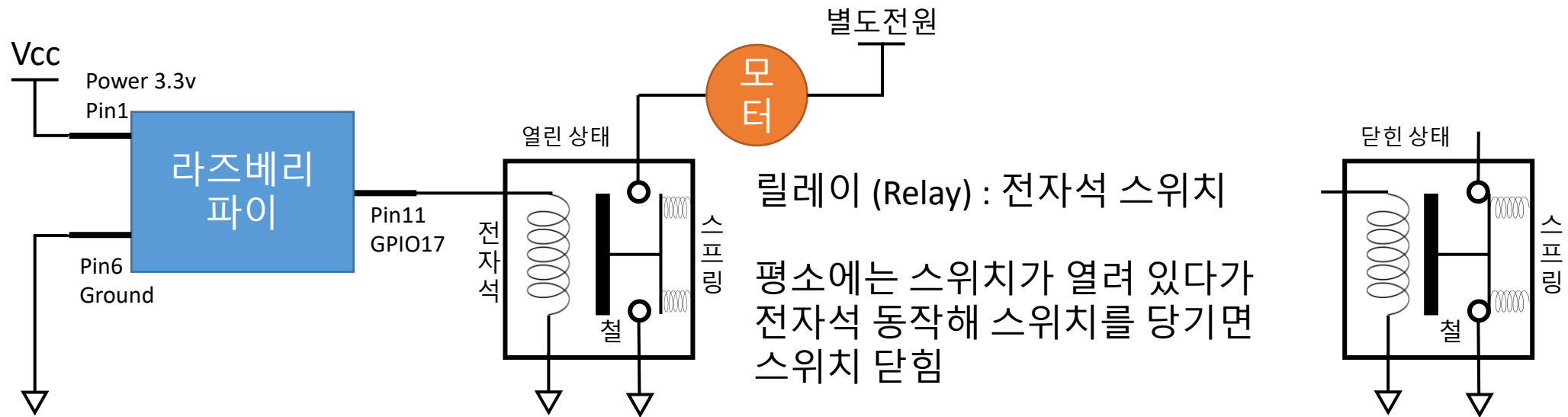
- 소형의 LED를 간신히 켤 수 있는 정도, 큰 전류가 필요한 모터를 사용하면 SoC가 탈 수 있음



GPIO 출력의 활용방법 - 스위치 제어

GPIO 출력의 전류가 작으므로 다른 스위치를 제어하는데 사용

- 전기 신호를 받았을 때 스위치가 닫히는 소자 : 릴레이, FET(Field Effect Transistor) 등



➤ 단순한 켜고 끄는 것 대신 강도를 조정할 수는 없을까? PWM (Pulse Width Modulation)



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

Chapter. 2

새로운 GPIO 하드웨어 만들기

GPIO Chip 이란?

GPIO는 물리적인 특성도 고려해야 하므로 별도의 하드웨어로 구성됨

MMIO를 통해 각각의 GPIO 핀을 설정할 수 있음

- GPIO 핀은 설정에 따라 입력 신호를 받을 수도 있고 출력 신호를 보낼 수도 있음
 - 출력 신호를 High로 보낼지 Low로 보낼지 선택 가능
 - 입력 신호가 High인지 Low인지 읽을 수 있음

입력 신호를 통해 인터럽트를 발생시키기도 함

- 레벨 트리거 (Level Trigger) : 특정 값이면 인터럽트를 무한히 발생시킴
- 엣지 트리거 (Edge Trigger) : 값이 변하면 인터럽트를 발생시킴
 - 라이징 엣지 (Rising Edge) : Low 에서 High로 변할 때
 - 폴링 엣지 (Falling Edge) : High 에서 Low로 변할 때

QEMU의 새 SoC에 GPIO Chip 추가하기

```
...
struct ComentoMachineState {
    ...
    DeviceState *gpio;
};
...
#define COMENTO_IRQ_GPIO 4
...
#define COMENTO_BASE_GPIO      0x09014000
...

static void create_gpio(ComentoMachineState *vms, MemoryRegion *mem)
{
    hwaddr base = COMENTO_BASE_GPIO;
    int irq = COMENTO_IRQ_GPIO;
    SysBusDevice *s;

    // ARM PL061 GPIO chip 장치 추가
    vms->gpio = qdev_new("pl061");

    // GPIO 핀 내부에 풀업 저항 대신 풀다운 저항이 달려 있다고 가정
    qdev_prop_set_uint32(vms->gpio, "pullups", 0);
    qdev_prop_set_uint32(vms->gpio, "pulldowns", 0xff);

    s = SYS_BUS_DEVICE(vms->gpio);
    sysbus_realize_and_unref(s, &error_fatal);
    memory_region_add_subregion(mem, base,
                                sysbus_mmio_get_region(s, 0));
}
```

```
// GPIO 핀의 값이 변했을 때 발생할 인터럽트 (레벨 트리거, 엣지 트리거)
sysbus_connect_irq(s, 0, qdev_get_gpio_in(vms->gic, irq));
}

...
static void machcomento_init(MachineState *machine)
{
    ...
    create_gpio(vms, sysmem);
}
```

<hw/arm/comento.c>

디바이스 트리와 커널 설정에 GPIO Chip 추가하기

```
...
gpio: pl061@9012000 {
    // 지금 명시하는 장치가 GPIO chip임을 명시
    gpio-controller;
    compatible = "arm,pl061","arm,primecell";
    // GPIO chip의 인터럽트와 MMIO 주소 명시
    interrupts = <GIC_SPI 4 IRQ_TYPE_LEVEL_HIGH>;
    reg = <0x00 0x9014000 0x00 0x1000>;
    // AMBA 장치이므로 클럭 필요
    clock-names = "apb_pclk";
    clocks = <&clock>;
    // GPIO chip을 참조할 때 숫자는 2칸 사용
    // 첫 번째 칸은 어떤 핀을 참조할지
    // 두 번째 칸은 값을 설정할 때 1이 High인지 low인지 설정
    // 일반적으로 1은 High를 의미하므로 GPIO_ACTIVE_HIGH로 설정
    #gpio-cells = <2>;
};
...
```

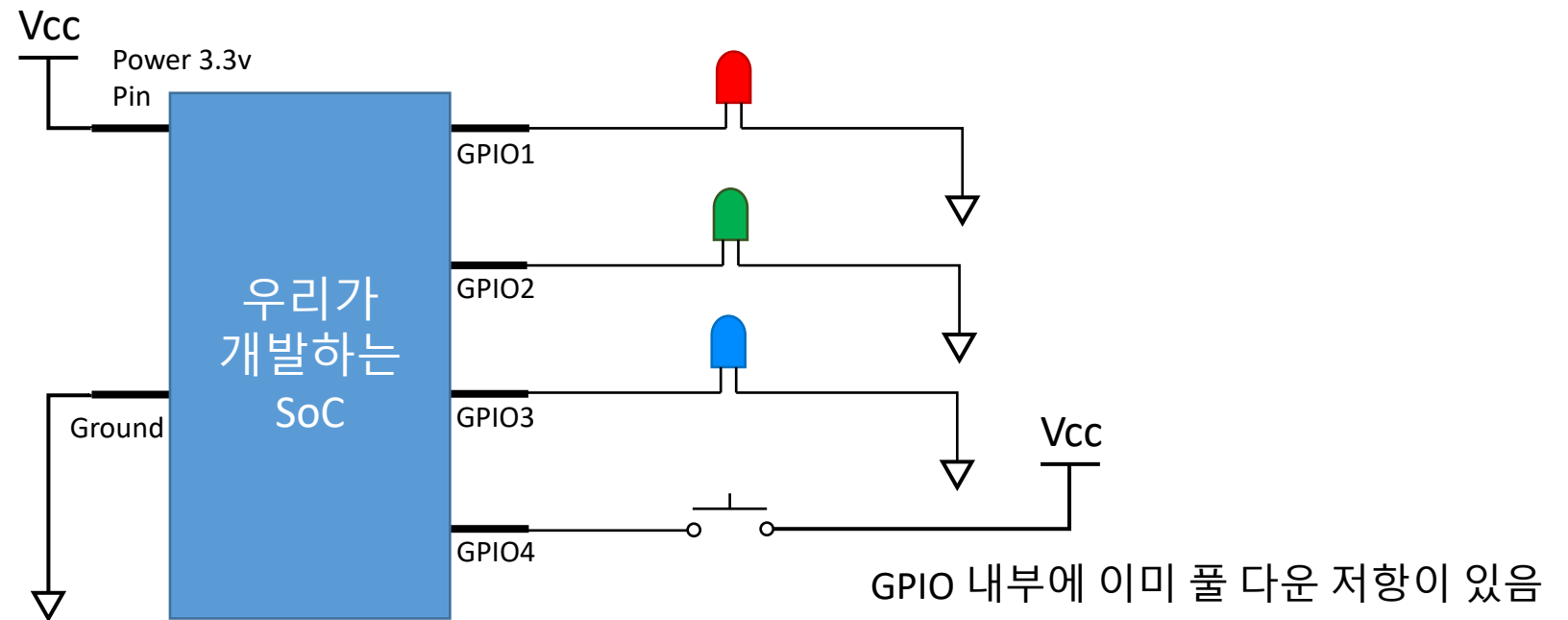
<arch/arm64/boot/dts/comento/comento.dts>

```
...
CONFIG_GPIO_PL061=y
# libgpiod를 사용하기 위해서 필요한 설정
CONFIG_GPIOLIB=y
# sysfs를 사용하는 옛날 방식으로 GPIO를 사용하기 위한 설정
CONFIG_GPIO_SYSFS=y
# GPIO_SYSFS는 일반적인 설정이 아니므로 전문가 사용자를 위한 설정 필요
CONFIG_EXPERT=y
```

<arch/arm64/configs/comento_defconfig>

수정한 defconfig를 적용하기 위해
make comento_defconfig 명령어 사용

수업에서 사용할 GPIO의 시나리오



- 위 회로와 비슷하게 GPIO 사용 하드웨어를 QEMU로 에뮬레이션 해봅시다.
 - LED를 켜거나 끌 때는 해당 색상의 글자로 "LED ON/OFF" 출력
 - 버튼 누름은 QMP의 key 속성에 0(안누름) 또는 1(누름) 입력

QEMU에 GPIO를 사용하는 하드웨어 추가하기 (1/2)

```
#include "qemu/osdep.h"
#include "hw/sysbus.h"
#include "hw/irq.h"
#include "sysemu/runstate.h"
#include "qapi/visitor.h"
#include "chardev/char-fe.h"

#define TYPE_GPIO_COMENTO "leds-and-button"
OBJECT_DECLARE_SIMPLE_TYPE(GPIO_COMENTO_State, GPIO_COMENTO)

struct GPIO_COMENTO_State {
    SysBusDevice parent_obj;
    int64_t key_state;
    qemu_irq key_pin;
};

// GPIO chip에서 보낸 신호가 level로 들어옴 (High - 1, Low - 0)
static void gpio_comento_handler(void *opaque, int n, int level)
{
    // 0 : 검정, 1 : 빨강, 2 : 초록, 3 : 노랑, 4: 파랑
    if (++ n == 3) {
        n = 4;
    }
    if (level) {
        // 쉘에서 색상을 출력하기 위한 방법 이용
        // \033[0;3<색상> : 해당 색상으로 설정, \033[0m : 원복
        printf("\033[0;3%dmLED ON\033[0m\n", n + 1);
    } else {
        printf("\033[1;3%dmLED OFF\033[0m\n", n + 1);
    }
}
```

```
// QMP로 key 속성을 얻어오려고 할 때 문자열로 반환
static char* gpio_comento_get_key(Object *obj, Error **errp)
{
    GPIO_COMENTO_State *s = GPIO_COMENTO(obj);
    return s->key_state? g_strdup("True") : g_strdup("False");
}

// QMP로 key 속성을 문자열로 넣었을 때 GPIO 핀 설정
static void gpio_comento_set_key(Object *obj, const char *value,
    Error **errp)
{
    GPIO_COMENTO_State *s = GPIO_COMENTO(obj);
    if (g_strcmp0(value, "true") && g_strcmp0(value, "True")) {
        s->key_state = 0;
        qemu_irq_lower(s->key_pin); // GPIO 핀을 High로 설정
    } else {
        s->key_state = 1;
        qemu_irq_raise(s->key_pin); // GPIO 핀을 Low로 설정
    }
}

static void gpio_comento_init(Object *obj)
{
    GPIO_COMENTO_State *s = GPIO_COMENTO(obj);
    DeviceState *dev = DEVICE(obj);
    // 0번 부터 3개의 핀(0 ~ 2)은 GPIO 출력 핀 (하드웨어 입장에서서는 입력)
    qdev_init_gpio_in(dev, gpio_comento_handler, 3);
    // 3 부터 1개의 핀(3)은 GPIO 입력 핀 (하드웨어 입장에서서는 출력)
    qdev_init_gpio_out(dev, &s->key_pin, 1);
    s->key_state = 0;
    object_property_add_str(obj, "key",
        gpio_comento_get_key, gpio_comento_set_key);
}
```

QEMU에 GPIO를 사용하는 하드웨어 추가하기 (2/2)

```
static const TypeInfo gpio_comento_info = {
    .name          = TYPE_GPIO_COMENTO,
    .parent        = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(GPIO_COMENTO_State),
    .instance_init = gpio_comento_init,
};
```

```
static void gpio_comento_register_types(void)
{
    type_register_static(&gpio_comento_info);
}
```

```
type_init(gpio_comento_register_types)
```

<hw/misc/comento/gpio.c>

```
...
softmmu_ss.add(when: 'CONFIG_COMENTO', if_true: files('gpio.c'))
```

<hw/misc/comento/meson.build>

```
...
static void create_leds_and_button(ComentoMachineState *vms)
{
    DeviceState *dev;
    char name[] = "leds-and-button";
    dev = qdev_new(name); // hw/misc/comento/gpio.c에 추가한 하드웨어 생성
    qdev_set_id(dev, name, NULL); // QMP 스크립트에서 사용할 이름 지정
    sysbus_realize_and_unref(SYS_BUS_DEVICE(dev), &error_fatal);
    // GPIO의 0~3번 핀을 하드웨어에 연결
    qdev_connect_gpio_out(vms->gpio, 0, qdev_get_gpio_in(dev, 0));
    qdev_connect_gpio_out(vms->gpio, 1, qdev_get_gpio_in(dev, 1));
    qdev_connect_gpio_out(vms->gpio, 2, qdev_get_gpio_in(dev, 2));
    qdev_connect_gpio_out(dev, 0, qdev_get_gpio_in(vms->gpio, 3));
}
```

```
...
static void machcomento_init(MachineState *machine)
{
    ...
    create_leds_and_button(vms);
}
```

<hw/arm/comento.c>

1. hw/misc/comento/에 GPIO를 사용하는 "leds-and-button" 하드웨어 추가
 2. hw/arm/comento.c에 해당 하드웨어를 추가하고 GPIO chip과 연결
- 추가한 GPIO chip과 하드웨어 (3개 LED & 1개 버튼)를 어떻게 테스트 할 수 있을까요?

Rootfs에 libgpiod와 libgpiod 툴 추가하기

GPIO를 사용하기 위해서는 커널에서 제공하는 GPIOLIB 인터페이스 사용해야 함

libgpiod는 GPIOLIB 인터페이스를 사용을 도와주는 라이브러리

- 리눅스에서 관리 : <https://git.kernel.org/pub/scm/libs/libgpiod/libgpiod.git/about/>
- 커맨드 라인에서 편리하게 사용할 수 있는 툴 제공

Rootfs에 libgpiod를 추가할 수 있도록 빌드루트 설정

- make menuconfig
 - Target packages -> Libraries -> Hardware handling -> libgpiod : 선택
 - Target packages -> Libraries -> Hardware handling -> libgpiod -> install tools : 선택

빌드 루트 빌드하기

- make -j<코어 개수>

Roofs를 SD 카드 이미지로 복사하기 (1/2)

1. 이미지 파일을 Loop 디바이스로 설정

- 이미지 파일을 바로 마운트 불가, Loop 디바이스 상태에서 마운트 가능
- `sudo losetup -Pf --show sdcard.img`

```
mentoring@comento:~$ sudo losetup -Pf --show sdcard.img  
/dev/loop8
```

- 출력되는 디바이스 노드 파일이 설정된 loop 디바이스

2. Loop 디바이스의 첫 번째 파티션을 ext4 형식으로 포맷

- `sudo mkfs.ext4 <loop 디바이스 경로>p1`

```
mentoring@comento:~$ sudo mkfs.ext4 /dev/loop8p1  
mke2fs 1.46.5 (30-Dec-2021)  
Discarding device blocks: done  
Creating filesystem with 16128 4k blocks and 16128 inodes  
  
Allocating group tables: done  
Writing inode tables: done  
Creating journal (1024 blocks): done  
Writing superblocks and filesystem accounting information: done
```

Rootfs를 SD 카드 이미지로 복사하기 (2/2)

3. 포맷한 파티션과 빌드루트에서 생성된 이미지 동시에 마운트

- `mkdir mnt1 mnt2`
- `sudo mount -o loop <loop 디바이스 경로>p1 mnt1`
- `sudo mount -o loop <빌드루트 디렉토리>/output/images/rootfs.ext4 mnt2`

4. 마운트된 빌드루트 이미지의 내용을 마운트된 포맷 파티션으로 복사

- `sudo cp -R mnt2/* mnt1/.`

5. 마운트한 디렉토리를 모두 언마운트

- `sync; sudo umount mnt1 mnt2`

6. Loop 디바이스 해제

- `sudo losetup -d <loop 디바이스 경로>`



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 3

libgpiod 툴로 GPIO 테스트 하기

sysfs를 사용하는 옛날 GPIO 제어 방식

많이 알려져 있는 GPIO 제어 방식이나 2015년부터 비공식지원으로 바뀜

/sys/class/gpio/

- gpiochip<숫자> : GPIO chip 관련 정보를 담고 있는 디렉토리
- gpio 파일 숫자는 gpiochip의 숫자 + 핀 번호로 지정
- export : 받은 숫자에 해당하는 gpio<숫자> 파일 생성
- unexport : 받은 숫자에 해당하는 gpio<숫자> 파일 제거

/sys/class/gpio/gpio<숫자>/

- direction : GPIO 핀을 입력(in)으로 사용할지 출력(out)으로 사용할지 결정
- value : GPIO 핀의 현재 값

sysfs를 사용하는 GPIO 테스트 실습 (1/3)

```
# cd /sys/class/gpio/
# ls
export      gpiochip512  unexport
# echo 512 > export
# echo 513 > export
# echo 514 > export
# echo 515 > export
# ls
export      gpio513      gpio515      unexport
gpio512     gpio514      gpiochip512
# ls gpio512
active_low  direction  power        uevent
device      edge        subsystem    value
# echo out > gpio512/direction
# echo out > gpio513/direction
# echo out > gpio514/direction
# echo in > gpio515/direction
# cat gpio512/direction
out
# cat gpio513/direction
out
# cat gpio514/direction
out
# cat gpio515/direction
in
```

- 존재하는 gpiochip 확인

- gpiochip512였으므로 GPIO 핀 지정할 때 512를 더함
- GPIO 핀 0~3을 나타내는 512~515 생성

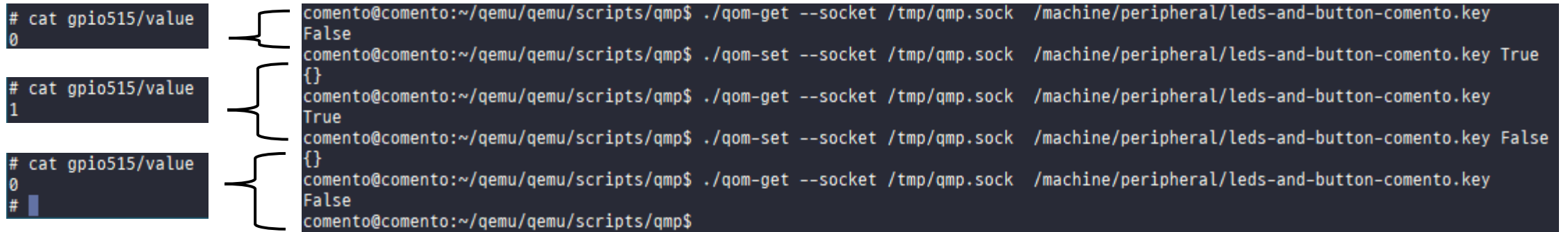
- GPIO 0~2는 출력, GPIO 3은 입력으로 설정

sysfs를 사용하는 GPIO 테스트 실습 (2/3)

```
# cat gpio512/value
0
# echo 1 > gpio512/value
LED ON
# echo 1 > gpio513/value
LED ON
# echo 1 > gpio514/value
LED ON
# cat gpio512/value
1
# echo 1 > gpio515/value
sh: write error: Operation not permitted
# echo 0 > gpio512/value
LED OFF
# echo 0 > gpio513/value
LED OFF
# echo 0 > gpio514/value
LED OFF
# cat gpio512/value
0
#
```

- GPIO 핀의 초기 값은 0으로 되어 있음
- GPIO 핀의 값을 1로 설정하면, GPIO 핀의 상태가 High로 바뀜
 - 0~2 핀에 할당된 대로 RGB LED가 순차적으로 켜짐
- 출력 GPIO핀은 sysfs로 값을 설정할 수 없음
 - Operation not permitted 에러 발생
- 값을 0으로 하면, 상태가 GPIO 핀이 Low로 바뀜

sysfs를 사용하는 GPIO 테스트 실습 (3/3)



```
# cat gpio515/value
0

# cat gpio515/value
1

# cat gpio515/value
0
#
```

```
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-get --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key
False
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key True
{}
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-get --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key
True
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key False
{}
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-get --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key
False
comento@comento:~/qemu/qemu/scripts/qmp$
```

- 기본값은 key가 눌리지 않은 False, GPIO 핀은 Low 상태임
- sysfs로 값을 읽으면 Low임을 의미하는 0
- QMP로 key 값을 True로 변경하면, GPIO 핀은 High 상태가 됨
- sysfs로 값을 읽으면 High임을 의미하는 1
- QMP로 key 값을 False로 변경하면, GPIO 핀은 Low 상태가 됨
- sysfs로 값을 읽으면 Low임을 의미하는 0

libgpiod 툴 사용하여 GPIO 테스트 하기

커널에서 제공하는 GPIOLIB 인터페이스를 사용하여 GPIO 제어

- gpio-detect : 존재하는 GPIO chip 탐색
- gpioinfo : 각 GPIO 핀 별로 현재 상태를 출력
- gpioget <GPIO chip 이름> <핀번호> : 해당 핀의 값을 출력
- gpioset <GPIO chip 이름> <핀번호>=<값> : 해당 핀의 값을 설정 (High = 1, Low = 0)
- gpiomon <GPIO chip 이름> <핀번호>.. : 지정한 핀들의 값을 모니터링
- 값이 바뀌면 (엣지 트리거), 어떤 종류의 엣지인지 출력

libgpiod 툴을 사용하는 GPIO 테스트 실습 (1/4)

```
# gpiodetect
gpiochip0 [9014000.pl061] (8 lines)
# gpioinfo
gpiochip0 - 8 lines:
    line 0:      unnamed      "sysfs"  input  active-high [used]
    line 1:      unnamed      "sysfs"  input  active-high [used]
    line 2:      unnamed      "sysfs"  input  active-high [used]
    line 3:      unnamed      "sysfs"  input  active-high [used]
    line 4:      unnamed      unused    input  active-high
    line 5:      unnamed      unused    input  active-high
    line 6:      unnamed      unused    input  active-high
    line 7:      unnamed      unused    input  active-high
# echo 512 > unexport
# echo 513 > unexport
# echo 514 > unexport
# echo 515 > unexport
# gpioinfo
gpiochip0 - 8 lines:
    line 0:      unnamed      unused    input  active-high
    line 1:      unnamed      unused    input  active-high
    line 2:      unnamed      unused    input  active-high
    line 3:      unnamed      unused    input  active-high
    line 4:      unnamed      unused    input  active-high
    line 5:      unnamed      unused    input  active-high
    line 6:      unnamed      unused    input  active-high
    line 7:      unnamed      unused    input  active-high
```

• 존재하는 gpiochip 확인

• gpiochip의 현재 상태 확인하기

- 방금 sysfs 실습 때문에 0~3 핀이 used 상태
- used 상태인 경우 libgpiod 툴로 제어 불가

• sysfs의 unexport로 gpio 파일 제거

• 제거한 이후에는 해당 핀의 used 상태가 풀림

libgpiod 툴을 사용하는 GPIO 테스트 실습 (2/4)

```
# gpioget gpiochip0 0
0
# gpioset gpiochip0 0=1
LED ON
# gpioset gpiochip0 1=1
LED ON
# gpioset gpiochip0 2=1
LED ON
# gpioget gpiochip0 0
LED OFF
1
# gpioset gpiochip0 0=1
LED ON
# gpioset gpiochip0 0=0
LED OFF
# gpioset gpiochip0 1=0
LED OFF
# gpioset gpiochip0 2=0
LED OFF
# gpioinfo
gpiochip0 - 8 lines:
    line 0:      unnamed      unused  output  active-high
    line 1:      unnamed      unused  output  active-high
    line 2:      unnamed      unused  output  active-high
    line 3:      unnamed      unused  input   active-high
    line 4:      unnamed      unused  input   active-high
    line 5:      unnamed      unused  input   active-high
    line 6:      unnamed      unused  input   active-high
    line 7:      unnamed      unused  input   active-high
#
```

• 기존 값은 0으로 설정됨

• GPIO 핀 0~2의 값을 High로 설정

• RGB LED가 순차적으로 켜짐

• 출력 GPIO 핀을 읽으려고 시도하면, GPIO를
입력용으로 바꾸고 값을 읽음

• 입력용으로 바뀌면서 값이 0으로 초기화 됨

• GPIO 핀 0~2의 값을 Low로 설정

• RGB LED가 순차적으로 꺼짐

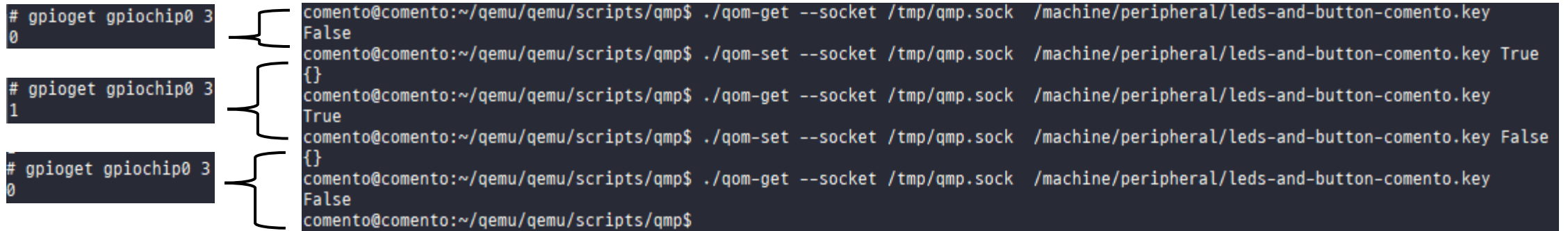
• GPIO 핀 0~2의 방향이 출력 방향으로 자동 설정

libgpiod 툴을 사용하는 GPIO 테스트 실습 (3/4)

```
# gpiodetect
gpiochip0 [9012000.pl061] (8 lines)
# gpioinfo
gpiochip0 - 8 lines:
    line 0:      unnamed      unused   input   active-high
    line 1:      unnamed      unused   input   active-high
    line 2:      unnamed      unused   input   active-high
    line 3:      unnamed      unused   input   active-high
    line 4:      unnamed      "spi0 CS0" output active-high [used]
    line 5:      unnamed      "spi0 CS1" output active-high [used]
    line 6:      unnamed      unused   input   active-high
    line 7:      unnamed      unused   input   active-high
# gpioset gpiochip0 1=1
LED ON
# gpioset gpiochip0 0=1
LED ON
# gpioinfo
gpiochip0 - 8 lines:
    line 0:      unnamed      unused   output  active-high
    line 1:      unnamed      unused   output  active-high
    line 2:      unnamed      unused   input   active-high
    line 3:      unnamed      unused   input   active-high
    line 4:      unnamed      "spi0 CS0" output active-high [used]
    line 5:      unnamed      "spi0 CS1" output active-high [used]
    line 6:      unnamed      unused   input   active-high
    line 7:      unnamed      unused   input   active-high
# gpioget gpiochip0 3
0
# gpioget gpiochip0 3
1
```

- 존재하는 gpiochip 확인
- gpiochip의 현재 상태 확인하기
 - 아직까지는 0, 1번핀의 상태가 input
- 0, 핀 1의 신호를 1로 설정
 - 각 핀에 연결되어 있는 LED가 켜짐
- 0, 핀 1의 상태가 output으로 변경됨
- GPIO에 어떤 신호가 들어오는지 조회

libgpiod 툴을 사용하는 GPIO 테스트 실습 (4/4)



```
# gpioget gpiochip0 3
0
# gpioget gpiochip0 3
1
# gpioget gpiochip0 3
0

comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-get --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key
False
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key True
{}
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-get --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key
True
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key False
{}
comento@comento:~/qemu/qemu/scripts/qmp$ ./qom-get --socket /tmp/qmp.sock /machine/peripheral/leds-and-button-comento.key
False
comento@comento:~/qemu/qemu/scripts/qmp$
```

- 기본값은 key가 눌리지 않은 False, GPIO 핀은 Low 상태임
- sysfs로 값을 읽으면 Low임을 의미하는 0
- QMP로 key 값을 True로 변경하면, GPIO 핀은 High 상태가 됨
- sysfs로 값을 읽으면 High임을 의미하는 1
- QMP로 key 값을 False로 변경하면, GPIO 핀은 Low 상태가 됨
- sysfs로 값을 읽으면 Low임을 의미하는 0

libgpiod 라이브러리를 사용한 프로그램 만들기

```
#include <gpiod.h> // libgpiod 헤더 파일을 include 함
#include <stdio.h>
#include <unistd.h>

int main() {
    // /dev에 생긴 GPIO chip의 디바이스 노드 파일 경로로 열기
    struct gpiod_chip *chip = gpiod_chip_open("/dev/gpiochip0");
    struct gpiod_line *line[4];
    int i;

    for (i = 0; i < sizeof(line) / sizeof(*line); i++) {
        // GPIO 핀 0~3을 얻어 오기
        line[i] = gpiod_chip_get_line(chip, i);
    }
    // GPIO 핀 0 ~ 2는 출력으로 설정하고 초기값은 1로 설정
    gpiod_line_request_output(line[0], NULL, 1);
    gpiod_line_request_output(line[1], NULL, 1);
    gpiod_line_request_output(line[2], NULL, 1);
    // GPIO 핀 3은 입력으로 설정
    gpiod_line_request_input(line[3], NULL);
    // GPIO 핀 3의 값을 읽어오기
    printf("The current key status - %d\n", gpiod_line_get_value(line[3]));

    sleep(5);

    // GPIO 핀 0~2의 값을 모두 0으로 설정
    gpiod_line_set_value(line[0], 0);
    gpiod_line_set_value(line[1], 0);
    gpiod_line_set_value(line[2], 0);
```

```
// GPIO 핀 3의 값을 읽어오기
printf("The current key status - %d\n", gpiod_line_get_value(line[3]));

return 0;
}
```

<gpio-sample.c>

- 빌드루트의 툴체인으로 프로그램 빌드
- <빌드루트디렉토리>/output/host/bin/aarch64-buildroot-linux-gnu-gcc gpio-sample.c -o gpio-sample -lgpiod
 - libgpiod 헤더와 라이브러리 경로 지정 필요 X
- SD 카드 이미지에 프로그램 복사
- `sudo losetup -Pf --show sdcard1.img`
- `sudo mount <loop 경로>p1 /mnt`
- `sudo cp gpio-sample /mnt/usr/bin/`
- `sync; sudo umount /mnt`
- `sudo losetup -d <loop 경로>`

libgpiod 라이브러리 사용 실습

```
comento@comento:~$ buildroot-2023.08/output/host/bin/aarch64-buildroot-linux-gnu-gcc gpio-sample.c -o gpio-sample -lgpiod
comento@comento:~$ sudo losetup -Pf --show sdcard.img
/dev/loop1
comento@comento:~$ sudo mount /dev/loop1p1 /mnt
comento@comento:~$ sudo cp gpio-sample /mnt/usr/bin/.
comento@comento:~$ sync
comento@comento:~$ sudo umount /mnt
comento@comento:~$ sudo losetup -d /dev/loop1
```

<빌드 및 SD 카드 이미지로 복사>

```
# gpio-sample
Hello
Line #0
Line #1
Line #2
Line #3
LED ON
LED ON
LED ON
The current key status - 0
LED OFF
LED OFF
LED OFF
The current key status - 1
```

<실행 결과>



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

Chapter. 4

드라이버에서 GPIO 사용하기

앞서 만든 드라이버에서 GPIO 연동 하기

드라이버에서도 사용자 프로그램에서 libgpiod를 썼던 것과 유사하게 GPIO 사용 가능

디바이스 트리로 사용할 GPIO를 명시하면 드라이버에서 사용 가능

- 다음과 같은 시나리오를 사용하여 MMIO 하드웨어 드라이버에서 GPIO를 연동해 봅시다
 - 데이터를 송신하는 동안 Red LED를 켜
 - 데이터를 수신하는 동안 Green LED를 켜
 - 수신한 데이터가 있으면 Blue LED를 켜
 - 버튼을 누르는 순간 수신한 데이터를 모두 지움

디바이스 트리에 사용할 GPIO 명시하기

<이름>-gpios, <이름>-gpio로 디바이스 트리에 명시 가능

- 이름 하나에 하나 이상의 GPIO 핀 지정 가능
- 드라이버에서는 명시한 이름으로 GPIO 핀을 얻어올 수 있음

```
...
#include <dt-bindings/gpio/gpio.h>
...
    mmio@9020000 {
        ...
        led-gpios = <&gpio 0 GPIO_ACTIVE_HIGH>,
                    <&gpio 1 GPIO_ACTIVE_HIGH>,
                    <&gpio 2 GPIO_ACTIVE_HIGH>;
        button-gpio = <&gpio 3 GPIO_ACTIVE_HIGH>;
    };
...
```

<arch/arm64/boot/dts/comento/comento.dts>

드라이버에서 GPIO 핀 값 설정하기

```
// GPIOLIB을 커널에서 사용하기 위해 추가한 헤더 파일
#include <linux/gpio/consumer.h>
...
struct comento_mmio_device {
    ...
    // gpio_descs는 여러 개의 GPIO 핀을 하나의 이름이 지정했을 때 사용
    // 멤버 변수 ndesc로 개수를, desc[<인덱스>]로 해당하는 핀 접근 가능
    struct gpio_descs *leds;
    // gpio_desc는 핀 하나를 의미
    struct gpio_desc *button;
};
...
static void comento_dma_tx_callback(void *data)
{
    ...
    // 송신 중인지를 알 수 있는 방법은 cmdev->tx.queued의 값을 보면 됨
    cmdev->tx.queued = false; // 송신 중이 아니므로,
    gpiod_set_value(cmdev->leds->desc[0], 0); // 핀 0을 Low로 변경
    ...
}
...
static ssize_t comento_mmio_write(struct file *fp, const char __user *buf,
                                size_t len, loff_t *ppos)
{
    ...
    cmdev->tx.queued = true; // 송신 중이므로,
    gpiod_set_value(cmdev->leds->desc[0], 1); // 핀 0을 High로 변경
    ...
}
...
```

```
static void comento_dma_rx_callback(void *data)
{
    ...
    // 수신 중인지를 알 수 있는 방법은 cmdev->rx.queued의 값을 보면 됨
    cmdev->rx.queued = false; // 수신 중이 아니므로,
    gpiod_set_value(cmdev->leds->desc[1], 0); // 핀 1을 Low로 변경
    ...
}
...
static void comento_mmio_rx_work_handler(struct work_struct *work)
{
    ...
    cmdev->tx.queued = true; // 수신 중임으로,
    gpiod_set_value(cmdev->leds->desc[0], 1); // 핀 1을 High로 변경
    ...
}
...
static void comento_mmio_rx_dma_work_handler(struct work_struct *work)
{
    ...
    // rx_size를 보면 현재 수신한 데이터가 있는지 알 수 있음
    cmdev->rx_size = cmdev->rx.sg.length - state.residue;
    if (cmdev->rx_size != 0) { // 수신한 데이터가 있다면,
        gpiod_set_value(cmdev->leds->desc[2], 1); // 핀 2을 High로 변경
    }
    ...
}
...
```

<drivers/comento/mmio.c>

드라이버에서 GPIO 핀 값 변화 체크하기

```
...
// GPIO 핀의 값이 변하는지 체크 하는 것은 인터럽트 처리처럼 구현
// 인터럽트 핸들러를 구현하여 설정하면, GPIO 핀 값에 따라 핸들러가 호출됨
static irqreturn_t comento_button_interrupt(int irq, void *dev_id)
{
    struct comento_mmio_device *cmdev = dev_id;

    // rx_size를 변경해야 하므로 spinlock으로 보호 필요
    write_lock(&cmdev->lock);

    // 인터럽트 핸들러가 라이징 엣지인 상황에 호출되면,
    // 수신한 데이터가 없는 것처럼 rx_size를 0으로 설정
    cmdev->rx_size = 0;
    // 수신할 데이터가 없으므로 핀 2을 LOW로 설정
    gpiod_set_value(cmdev->leds->desc[2], 0);

    write_unlock(&cmdev->lock);

    // 처리가 완료 되었음을 반환
    return IRQ_HANDLED;
}

...

static int comento_mmio_probe(struct amba_device *adev,
                             const struct amba_id *id)
{
    ...
}
```

```
// 디바이스 트리에서 led-gpios로 설정된 GPIO 핀들을 배열로 얻어옴
// 해당 GPIO 핀은 출력용으로 사용하며 초기 값은 Low로 하도록 설정
cmdev->leds = devm_gpiod_get_array(&adev->dev, "led", GPIOD_OUT_LOW);

// 디바이스 트리에서 button-gpios로 설정된 GPIO 핀을 얻어옴
// 해당 GPIO 핀은 입력용으로 사용하도록 설정
cmdev->button = devm_gpiod_get(&adev->dev, "button", GPIOD_IN);

// 입력용 GPIO 핀의 값 변화를 체크하기 위해 인터럽트로 변경
irq = gpiod_to_irq(cmdev->button);

// GIC에서 발생하는 인터럽트처럼
// GPIO 핀의 변화 발생시 핸들러가 호출됨
ret = devm_request_irq(&adev->dev, irq, comento_button_interrupt,
    // 언제 핸들러가 호출될지를 지정, 여기서는
    // 라이징 엣지일 때 핸들러가 호출되도록 설정
    IRQF_TRIGGER_RISING,
    "mmio-comento-gpio", cmdev);

// 라이징 엣지 트리거 이외에도
// 폴링 엣지 트리거, 하이 레벨 트리거, 로우 레벨 트리거 사용 가능

// 하이 레벨이나 로우 레벨인 경우
// 값이 변하지 않으면 계속 핸들러가 호출되므로,
// 한번만 실행되도록 IRQF_ONESHOT을 같이 사용함
...
}
```

<drivers/comento/mmio.c>

드라이버로 GPIO 제어 확인 실습

```
# echo "Nice to meet you" > /dev/comento-mmio0  
LED ON  
LED OFF
```

```
comento@coment:~/linux-6.5.3$ ../qemu/scripts/qmp/qom-get --socket /tmp/qmp.sock  
/machine/peripheral/mmio-comento.data  
Nice to meet you  
  
comento@coment:~/linux-6.5.3$ ../qemu/scripts/qmp/qom-set --socket /tmp/qmp.sock  
/machine/peripheral/mmio-comento.data Hello world~!  
{}
```

```
# LED ON  
LED ON  
LED OFF  
  
# cat /dev/comento-mmio0  
Hello world~!#
```

```
comento@coment:~/linux-6.5.3$ ../qemu/scripts/qmp/qom-set --socket /tmp/qmp.sock  
/machine/peripheral/leds-and-button-comento.key True  
{}
```

```
LED OFF  
  
# cat /dev/comento-mmio0  
#
```

• 리눅스에서 데이터를 보내면,
빨강 LED가 점멸

• QMP로 데이터를 보내면,
리눅스에서 데이터를 받으며
녹색 LED가 점멸

• 수신한 데이터가 있으므로,
파랑 LED가 켜짐

• 버튼을 누름

• 수신한 데이터가 지워지고,
파랑 LED가 꺼짐



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

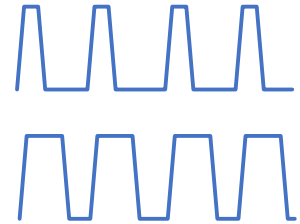
Chapter. 5

PWM 이해하기

PWM (Pulse Width Modulation) 이란?

GPIO의 출력으로 제어할 때는 High (On) 또는 Low (Off)만 제어 가능

- 켜고 끄고만 하는 게 아니라 세기를 조정할 수는 없을까?
 - 주기적으로 켜고 끄는 대신에 켜져 있는 비율을 조절하면 되지 않을까?
 - Ex1) 3ms 동안 켜고 7ms 동안 끈다면, 계속 켜는 것에 비해 30%의 세기만 나오지 않을까?
 - Ex2) 6ms 동안 켜고 4ms 동안 끈다면, 계속 켜는 것에 비해 60%의 세기만 나오지 않을까?



펄스의 폭을 변조하여 세기를 조정하는 방식

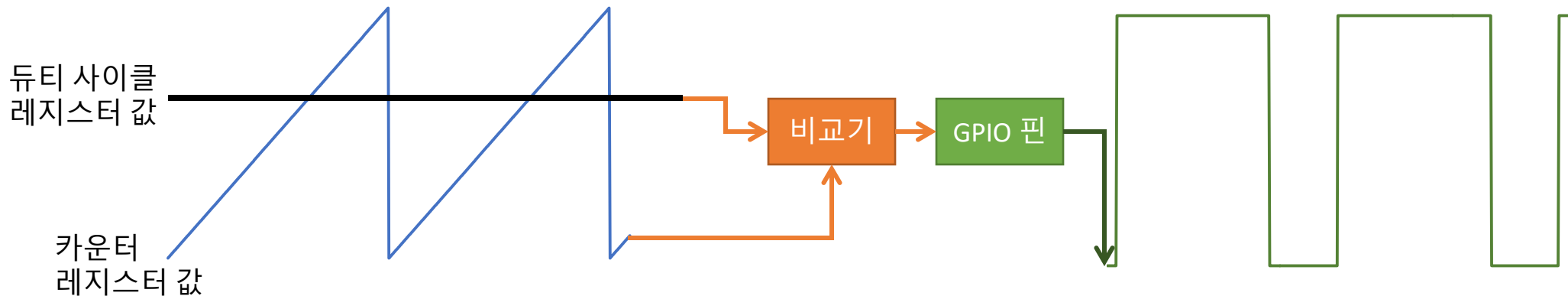
- 주기적으로 켜고 끄는 동작을 펄스라고 함. 펄스의 폭 중에서도 켜져 있는 폭을 변조 하는 것
 - 하나의 주기에서 켜져있는 시간의 비율을 듀티 사이클(Duty Cycle)이라고 함
 - 듀티 사이클 0%는 Low, 듀티 사이클 100%는 High 와 동일



PWM의 구조

듀티 사이클을 구현하기 위해 일반적으로 PWM은 다음과 같은 요소로 구현

- 한 클럭 당 값이 1씩 증가하는 카운터 레지스터
 - 끝에 다다르면 overflow되어 값이 0으로 바뀜
- 듀티 사이클을 지정할 수 있는 레지스터
- 레지스터 2개의 값을 비교하여 1 또는 0을 반환하는 GPIO 핀



PWM의 활용

1. LED / LCD 화면의 백라이트 / 전열기 / 모터 등의 기구 제어
 - 사람이 감지 할 수 없을 정도의 빠른 주파수 사용
 - 전열기나 모터 등 큰 전류를 요구하는 경우는 별도의 회로 설계 요구 됨
2. 원하는 전압으로 변환
 - 평탄한 전압과 원하는 전류를 사용할 수 있도록 회로 설계 요구됨
3. 아날로그 신호 출력 (DAC, Digital to Analog Converter)
 - 전압 변환 과정을 빠르게 하면 원하는 아날로그 신호 출력 가능
 - 오디오 신호에 사용

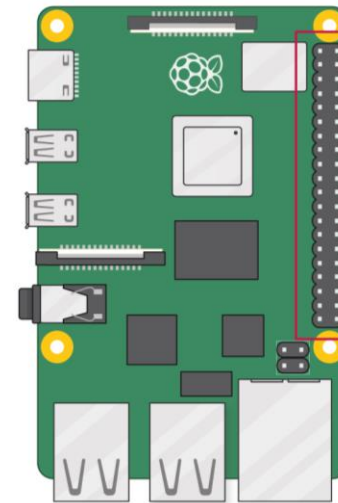
실제 SoC에서 PWM의 형태

라즈베리 파이에 사용되는 SoC(Broadcom社 BCM27xx)는 2개의 PWM 핀을 지원

- LCD의 백라이트 밝기 조절에 주로 사용 됨

외부로 나온 핀 헤더를 사용하여 PWM 사용 가능

- GPIO와 겸용으로 사용 되는 핀
 - PWM으로 사용하도록 사전 설정 필요



3V3 power	1	2	5V power
GPIO 2 (SDA)	3	4	5V power
GPIO 3 (SCL)	5	6	Ground
GPIO 4 (GPCLK0)	7	8	GPIO 14 (TXD)
Ground	9	10	GPIO 15 (RXD)
GPIO 17	11	12	GPIO 18 (PCM_CLK)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23
3V3 power	17	18	GPIO 24
GPIO 10 (MOSI)	19	20	Ground
GPIO 9 (MISO)	21	22	GPIO 25
GPIO 11 (SCLK)	23	24	GPIO 8 (CE0)
Ground	25	26	GPIO 7 (CE1)
GPIO 0 (ID_SD)	27	28	GPIO 1 (ID_SC)
GPIO 5	29	30	Ground
GPIO 6	31	32	GPIO 12 (PWM0)
GPIO 13 (PWM1)	33	34	Ground
GPIO 19 (PCM_FS)	35	36	GPIO 16
GPIO 26	37	38	GPIO 20 (PCM_DIN)
Ground	39	40	GPIO 21 (PCM_DOUT)

※ 출처 : 라즈베리 파이 문서

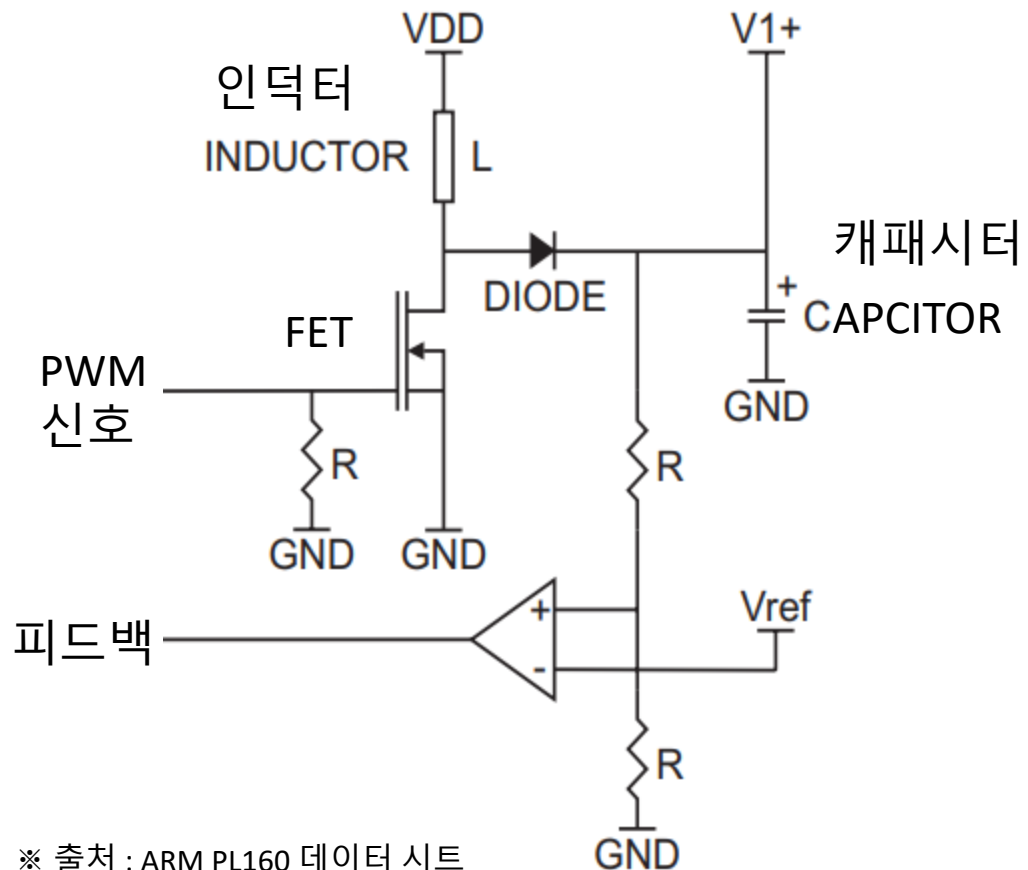
PWM 하드웨어 예시 - ARM PL160

PWM을 사용하여 2개의 핀 각각에 전압 변환을 하는 하드웨어

- 전자회로 내에서 전압 변환의 경우 DC to DC라고 부름

위치	이름	설명						
0x00	Drive 0 Configuration Register	듀티 사이클에 맞춰서 0번 핀에 전압을 출력 (0 - 0%, 15 – 100%)						
		<table><tr><th>비트</th><th>설명</th></tr><tr><td>0~3</td><td>DCDCDR0SEL 핀이 Low일 때 듀티 사이클</td></tr><tr><td>4~7</td><td>DCDCDR0SEL 핀이 High일 때 듀티 사이클</td></tr></table>	비트	설명	0~3	DCDCDR0SEL 핀이 Low일 때 듀티 사이클	4~7	DCDCDR0SEL 핀이 High일 때 듀티 사이클
		비트	설명					
		0~3	DCDCDR0SEL 핀이 Low일 때 듀티 사이클					
4~7	DCDCDR0SEL 핀이 High일 때 듀티 사이클							
0x04	Drive 1 Configuration Register	Drive 0 Configuration Register와 유사하게 1 번 핀에 적용						
0x08	Frequency Configuration Register	펄스의 주파수 설정						

PWM의 활용 – DC to DC 변환기



※ 출처 : ARM PL160 데이터 시트

- PWM 신호를 통해 VDD 전압을 듀티 사이클에 맞추어 V1 전압으로 변환
- PWM 신호 자체는 Low와 High로만 이루어짐
- 따라서 출력 역시 0V와 VDD로만 이루어짐
- 전압을 뭉뚱그려 평탄화할 필요가 있음
 - 인덕터, 캐패시터
- 물리적인 제어는 반드시 피드백이 요구됨
- 목표치에 도달하였는지 아닌지의 신호
- 피드백에 따라 듀티 사이클을 제어해야 함

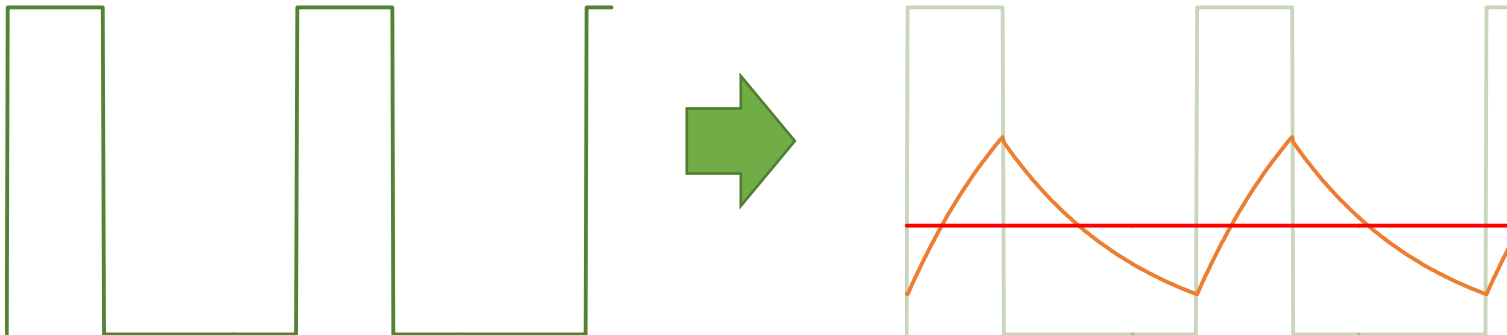
인덕터 (Inductor)와 캐피시터 (Capacitor)

신호를 평탄화 하기 위해서 적분의 성질을 가지고 있는 소자 이용

- 적분 : 인덕터 직렬 연결, 캐패시터 병렬 연결
- 인덕터 : 코일 소자으로써 자기장의 형태로 전류를 저장
- 캐패시터 : 넓은 금속판 2장 사이에 절연판 1장의 형태의 소자로 전압을 저장
- 전류와 전압이 있을 때 저장해 두었다가 없을 때 다시 꺼내어 쓰는 형태



원통 안에
얇은 판이
돌돌 말려
있음



왼쪽의 평균값 = 오른쪽 주황색 그래프의 평균값

※ 실제로는 빨간선 처럼 직선으로 평탄화되나 시연을 위해 계수를 조정함

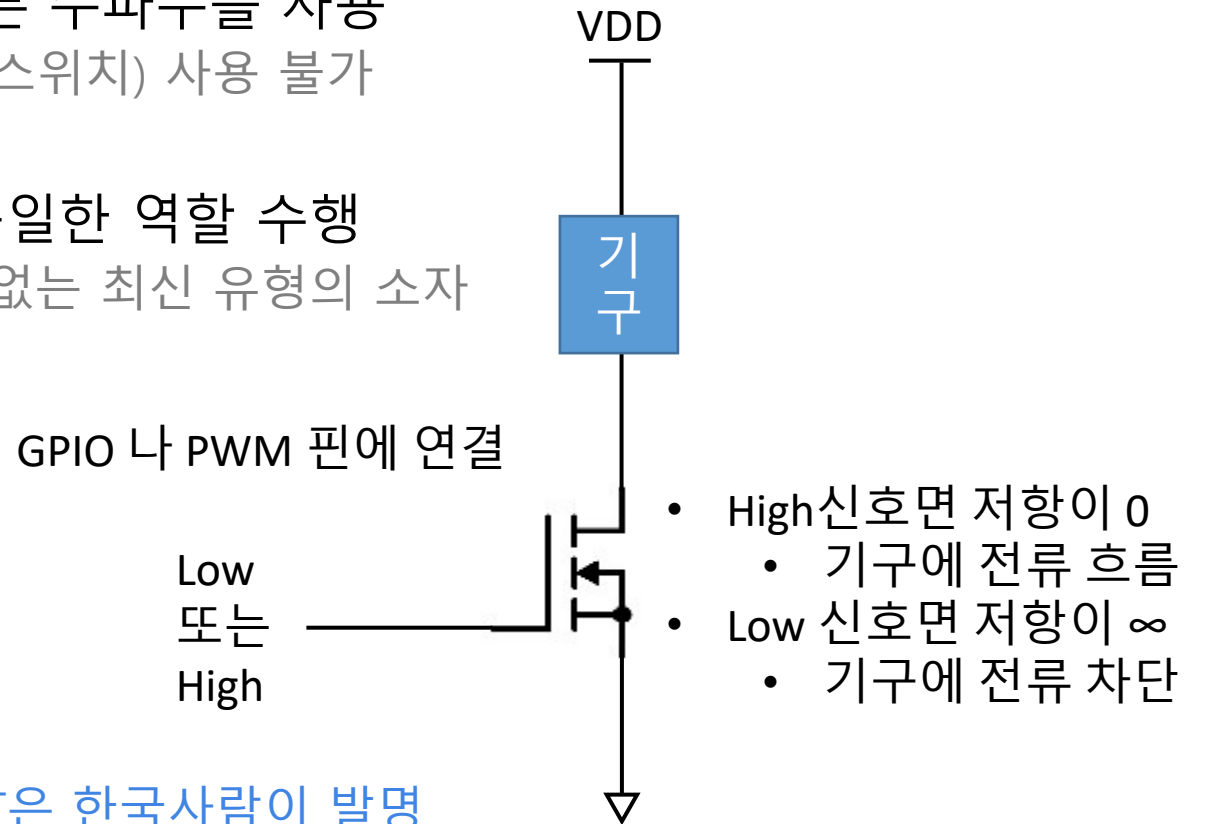
FET (Field Effect Transistor)

PWM의 경우는 사람이 눈치채지 못하게 빠른 주파수를 사용

- 물리적인 장치인 릴레이 (전자석으로 여닫는 스위치) 사용 불가

전자 소자인 트랜지스터는 릴레이와 거의 동일한 역할 수행

- 그 중에서도 FET는 스위치 단의 전류 소모가 없는 최신 유형의 소자



❖ 참고 : FET 중 가장 일반적인 형태인 MOS-FET은 한국사람이 발명



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

과제 안내

과제 의도

- GPIO를 사용하여 디바이스와 어떻게 통신할 수 있는지 살펴본다.
- GPIO와 PWM이 실제 환경에서 어떤 식으로 활용되는지 생각해 본다.

필수 과제 내용

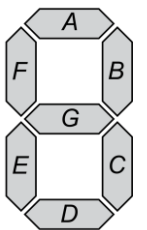
1. GPIO의 7번 라인을 입력으로 받아서 다음과 같은 기능을 구현한다.
 1. Low였다가 High로 바뀌는 경우, 앞서 Logging 디바이스에서 만드는 /tmp/qemu-mmio.log를 삭제한다.
2. 기능 구현을 위해 추가한 코드가 있다면 해당 파일을 업로드하여 제출 한다.

선택 과제 내용

1. 수업에서 진행했던 LED와 버튼 하드웨어를 수정하여 LED 7개로 바꾼다.
2. 실행시 숫자를 1개 입력 받은 뒤 숫자에 맞춰 7-segment LED를 켜는 프로그램을 작성한다.
 1. A는 핀0, B는 핀1, C는 핀2, D는 핀3, E는 핀4, F는 핀5, G는 핀6
3. 프로그램 코드를 업로드하여 제출한다.

기한 : 다음 주 월요일 오전 0시까지

<7-segment LED>



다음 주 커리큘럼

SPI 통신 이해하기

새로운 SPI 하드웨어 만들기

SPI 툴로 SPI 하드웨어 테스트하기

SPI 하드웨어 드라이버 만들기

ADC 이해하기

